PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies

Python source code, algorithms, assumptions, sample outputs, and conclusions

Student Name: K Hima Kailash

Roll Number: 26000924

Branch / Section: CSM-2

Sample policies and data are illustrative. Check faculty instructions and add required flowcharts and your own output screenshots before submission.

University Scholarship Allocation

Problem statement

Select candidates who meet aggregate, subject marks, attendance, and income criteria; rank eligible candidates.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store each student's data in a tuple and all students in a list.

2. 2. Check all eligibility conditions.

3. 3. Add eligible students to a list.

4. 4. Sort by aggregate and then attendance in descending order.

5. 5. Display the ranked list.

Python Source Code

```
# Case Study 1: University Scholarship Allocation
# Sample policy: aggregate >= 80, each subject >= 70, attendance >= 75,
# family income <= 300000. Rank eligible students by aggregate, then attendance.

students = [
    ("Asha", 88, [85, 90, 89], 92, 250000),
    ("Balu", 84, [78, 82, 80], 88, 280000),
    ("Charan", 91, [90, 68, 95], 95, 200000),
    ("Divya", 84, [80, 85, 87], 91, 290000),
    ("Esha", 79, [80, 82, 81], 90, 180000),
]

eligible = []
for name, aggregate, subjects, attendance, income in students:
    if (aggregate >= 80 and min(subjects) >= 70
            and attendance >= 75 and income <= 300000):
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda student: (student[1], student[2]), reverse=True)

print("ELIGIBLE SCHOLARSHIP CANDIDATES")
if eligible:
    for rank, student in enumerate(eligible, start=1):
        name, aggregate, attendance, income = student
        print(f"{rank}. {name} - Aggregate: {aggregate}%, Attendance: {attendance}%, Income: Rs.{income}")
else:
    print("No eligible candidates.")
```

Sample Output

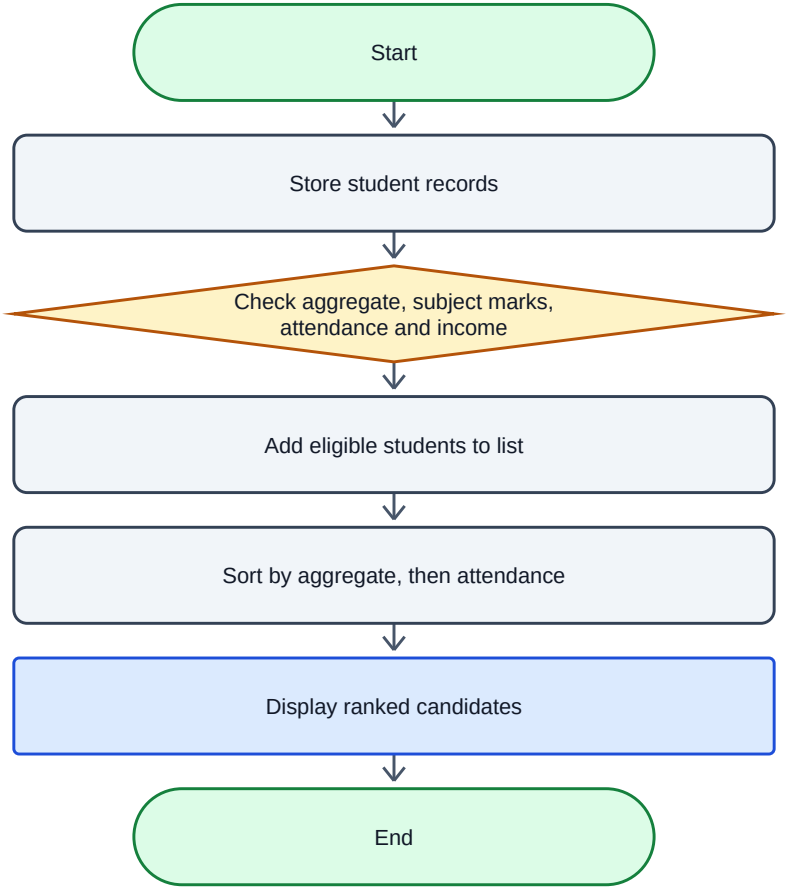
```
ELIGIBLE SCHOLARSHIP CANDIDATES
1. Asha - Aggregate: 88%, Attendance: 92%, Income: Rs.250000
2. Divya - Aggregate: 84%, Attendance: 91%, Income: Rs.290000
3. Balu - Aggregate: 84%, Attendance: 88%, Income: Rs.280000
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 1

University Scholarship Allocation



Decision branches are represented by the labelled outcomes in the flow.

Progressive Electricity Billing

Problem statement

Calculate monthly electricity bills using progressive slabs and identify large usage increases.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store monthly unit readings in a list of tuples.
2. 2. Calculate energy charges using if/elif/else slabs.
3. 3. Add the fixed charge.
4. 4. Compare each month with the previous month.
5. 5. Print bills and usage alerts.

Python Source Code

```
# Case Study 2: Progressive Electricity Billing
# Sample tariff: first 100 units Rs.1.50/unit; next 100 Rs.2.50/unit;
# above 200 Rs.4/unit; fixed charge Rs.50.

months = [("August", 145), ("September", 178), ("October", 235)]
previous_units = None

print("MONTHLY ELECTRICITY BILL")
for month, units in months:
    if units <= 100:
        energy_charge = units * 1.50
    elif units <= 200:
        energy_charge = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00

    total = energy_charge + 50
    print(f"{month}: {units} units, Bill = Rs.{total:.2f}")

    if previous_units is not None:
        increase = units - previous_units
        if increase > 30:
            print(f"  Alert: usage increased by {increase} units.")
    previous_units = units
```

Sample Output

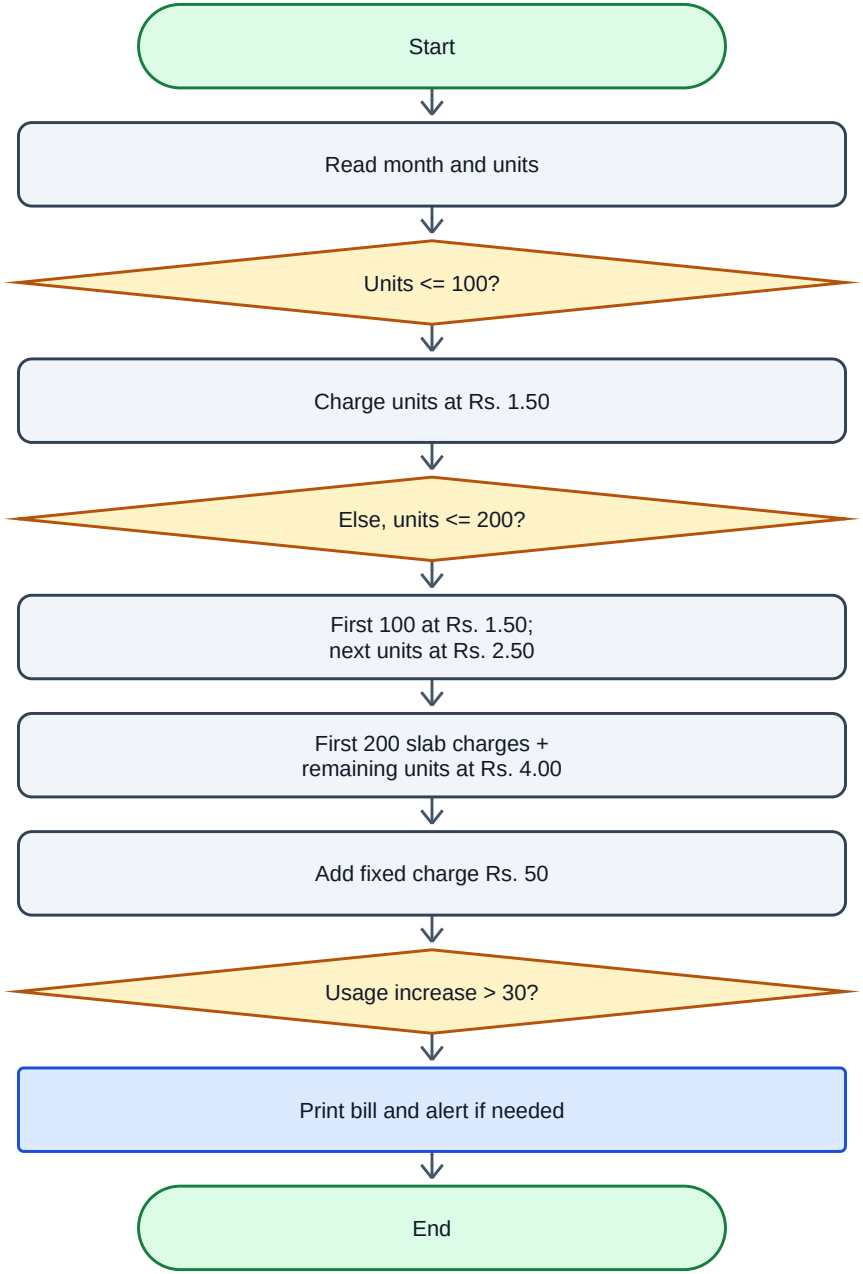
```
MONTHLY ELECTRICITY BILL
August: 145 units, Bill = Rs.312.50
September: 178 units, Bill = Rs.395.00
  Alert: usage increased by 33 units.
October: 235 units, Bill = Rs.590.00
  Alert: usage increased by 57 units.
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 2

Progressive Electricity Billing



Decision branches are represented by the labelled outcomes in the flow.

Retail Billing with Multiple Discount Rules

Problem statement

Generate an itemized bill, apply a threshold discount with a cap, and calculate tax.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store product name, unit price, and quantity in tuples.
2. 2. Calculate each line amount and subtotal.
3. 3. Apply discount when the threshold is met.
4. 4. Calculate tax on the discounted amount.
5. 5. Display the invoice.

Python Source Code

```
# Case Study 3: Retail Billing with Multiple Discount Rules
# Sample policy: 10% discount if subtotal >= Rs.1000; maximum discount Rs.200.
# Tax is applied after discount: 5% on all items for this example.

items = [("Notebook", 80, 5), ("Bag", 650, 1), ("Pen", 20, 10)]
subtotal = 0

print("ITEMIZED INVOICE")
for name, price, quantity in items:
    amount = price * quantity
    subtotal += amount
    print(f"{name}: {quantity} x Rs.{price} = Rs.{amount:.2f}")

discount = min(subtotal * 0.10, 200) if subtotal >= 1000 else 0
taxable_amount = subtotal - discount
tax = taxable_amount * 0.05
total = taxable_amount + tax

print(f"Subtotal: Rs.{subtotal:.2f}")
print(f"Discount: Rs.{discount:.2f}")
print(f"Tax (5%): Rs.{tax:.2f}")
print(f"Final total: Rs.{total:.2f}")
```

Sample Output

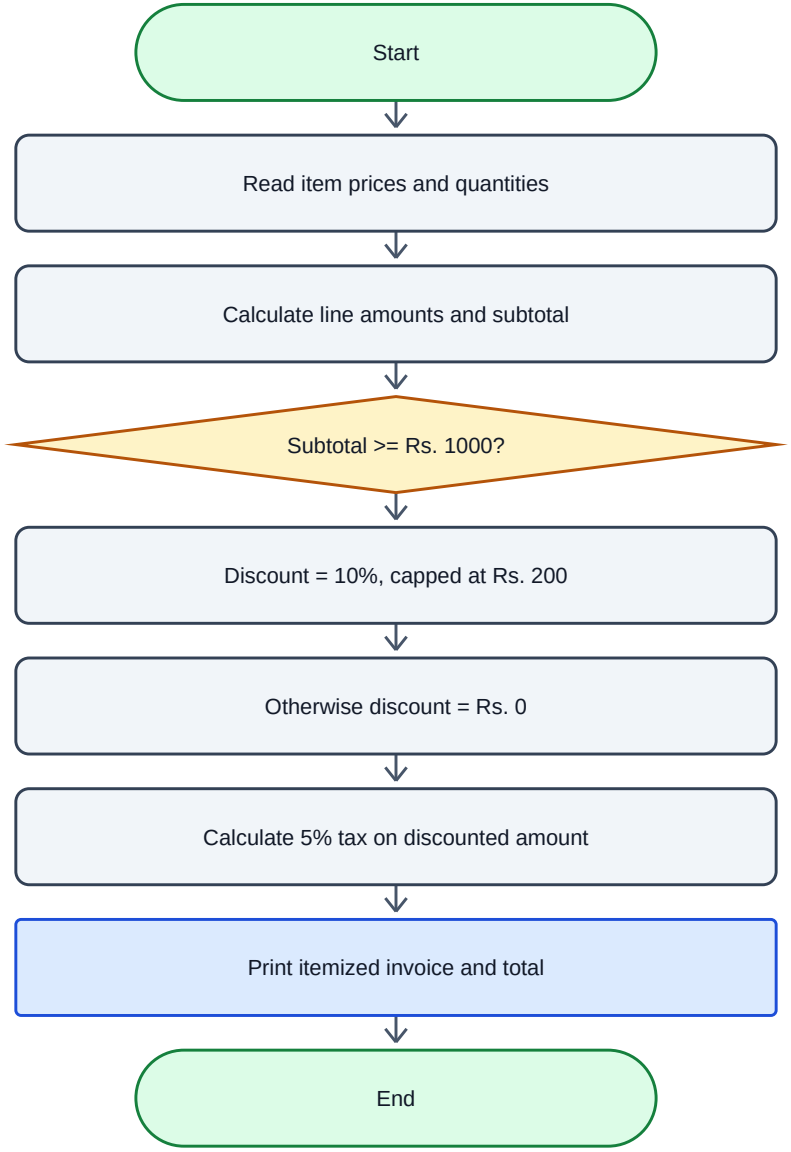
```
ITEMIZED INVOICE
Notebook: 5 x Rs.80 = Rs.400.00
Bag: 1 x Rs.650 = Rs.650.00
Pen: 10 x Rs.20 = Rs.200.00
Subtotal: Rs.1250.00
Discount: Rs.125.00
Tax (5%): Rs.56.25
Final total: Rs.1181.25
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 3

Retail Billing with Multiple Discount Rules



Decision branches are represented by the labelled outcomes in the flow.

Examination Result and Grace-Mark Validation

Problem statement

Apply a limited grace-mark policy and determine each student's final pass/fail status.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store each student's subject marks in a list.

2. 2. For marks from 35 to 39, add only the marks needed to reach 40.

3. 3. Keep all marks within 100.

4. 4. Check whether every subject reaches the pass mark.

5. 5. Print final marks, grace marks used, and result.

Python Source Code

```
# Case Study 4: Examination Result and Grace-Mark Validation
# Sample policy: pass mark is 40 in each subject. Up to 5 grace marks may be
# added to a subject scoring 35-39, without exceeding 100.

students = [
    ("Anu", [67, 72, 38]),
    ("Bharat", [55, 39, 61]),
    ("Chitra", [80, 44, 91]),
]

PASS_MARK = 40
MAX_GRACE = 5

print("EXAMINATION RESULTS")
for name, marks in students:
    final_marks = []
    grace_used = 0

    for mark in marks:
        if PASS_MARK - MAX_GRACE <= mark < PASS_MARK:
            add = min(MAX_GRACE, PASS_MARK - mark, 100 - mark)
            mark += add
            grace_used += add
        final_marks.append(mark)

    result = "PASS" if all(mark >= PASS_MARK for mark in final_marks) else "FAIL"
    print(f"{name}: Final marks = {final_marks}, Grace used = {grace_used}, Result = {result}")
```

Sample Output

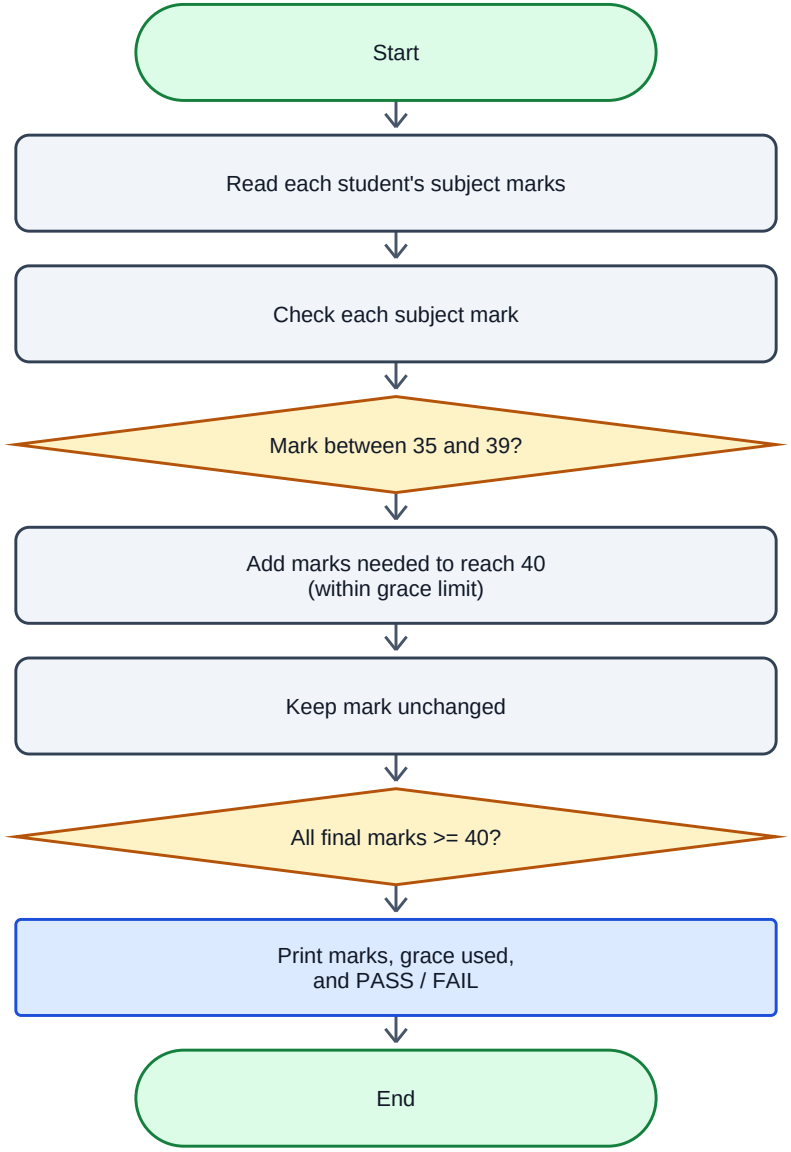
```
EXAMINATION RESULTS
Anu: Final marks = [67, 72, 40], Grace used = 2, Result = PASS
Bharat: Final marks = [55, 40, 61], Grace used = 1, Result = PASS
Chitra: Final marks = [80, 44, 91], Grace used = 0, Result = PASS
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 4

Examination Result and Grace-Mark Validation



Decision branches are represented by the labelled outcomes in the flow.

Warehouse Inventory Reconciliation

Problem statement

Compare expected and actual closing stock to detect shortages and surpluses.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store stock details in tuples.
2. 2. Calculate expected closing stock.
3. 3. Compare expected stock with actual stock.
4. 4. Calculate the difference.
5. 5. Report shortage, surplus, or reconciled stock.

Python Source Code

```
# Case Study 5: Warehouse Inventory Reconciliation
# Expected closing stock = opening stock + received - dispatched.

products = [
    ("Keyboard", 50, 20, 45, 23),
    ("Mouse", 80, 15, 30, 60),
    ("Monitor", 25, 10, 12, 23),
]

print("WAREHOUSE RECONCILIATION")
for name, opening, received, dispatched, actual_closing in products:
    expected = opening + received - dispatched
    difference = actual_closing - expected
    print(f"{name}: Expected={expected}, Actual={actual_closing}, Difference={difference}")
    if difference < 0:
        print(f"  Discrepancy: shortage of {-difference} units")
    elif difference > 0:
        print(f"  Discrepancy: surplus of {difference} units")
    else:
        print("  Stock reconciled.")
```

Sample Output

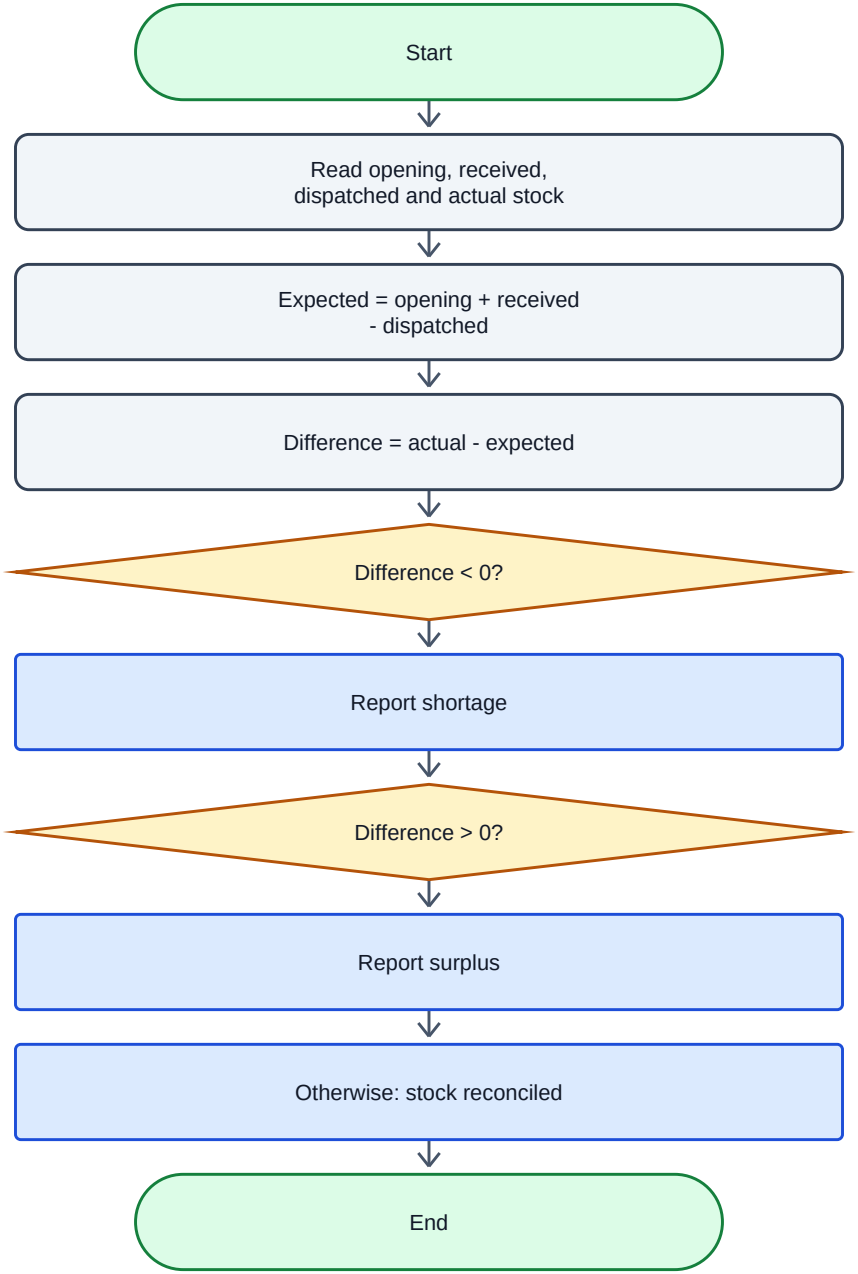
```
WAREHOUSE RECONCILIATION
Keyboard: Expected=25, Actual=23, Difference=-2
  Discrepancy: shortage of 2 units
Mouse: Expected=65, Actual=60, Difference=-5
  Discrepancy: shortage of 5 units
Monitor: Expected=23, Actual=23, Difference=0
  Stock reconciled.
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 5

Warehouse Inventory Reconciliation



Decision branches are represented by the labelled outcomes in the flow.

Student Timetable Conflict Detection

Problem statement

Detect overlapping class times and room conflicts using tuples and nested loops.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Represent each class as a tuple.
2. 2. Compare each pair of classes using nested loops.
3. 3. Check whether days match and time intervals overlap.
4. 4. Report a time conflict or a same-room conflict.
5. 5. Print the total number of conflicts.

Python Source Code

```
# Case Study 6: Student Timetable Conflict Detection
# Each tuple: (course, day, start_hour, end_hour, room)

classes = [
    ("Python", "Monday", 9, 10, "Lab-1"),
    ("Maths", "Monday", 10, 11, "Room-2"),
    ("Physics", "Monday", 9, 10, "Room-3"),
    ("English", "Tuesday", 9, 10, "Room-1"),
]

print("TIMETABLE CONFLICT CHECK")
conflicts = 0
for i in range(len(classes)):
    for j in range(i + 1, len(classes)):
        c1 = classes[i]
        c2 = classes[j]
        same_day = c1[1] == c2[1]
        overlap = c1[2] < c2[3] and c2[2] < c1[3]
        same_room = c1[4] == c2[4]
        if same_day and overlap:
            if same_room:
                print(f"Room conflict: {c1[0]} and {c2[0]} in {c1[4]}")
            else:
                print(f"Time conflict: {c1[0]} and {c2[0]} overlap on {c1[1]}")
            conflicts += 1

if conflicts == 0:
    print("No conflicts found.")
else:
    print(f"Total conflicts: {conflicts}")
```

Sample Output

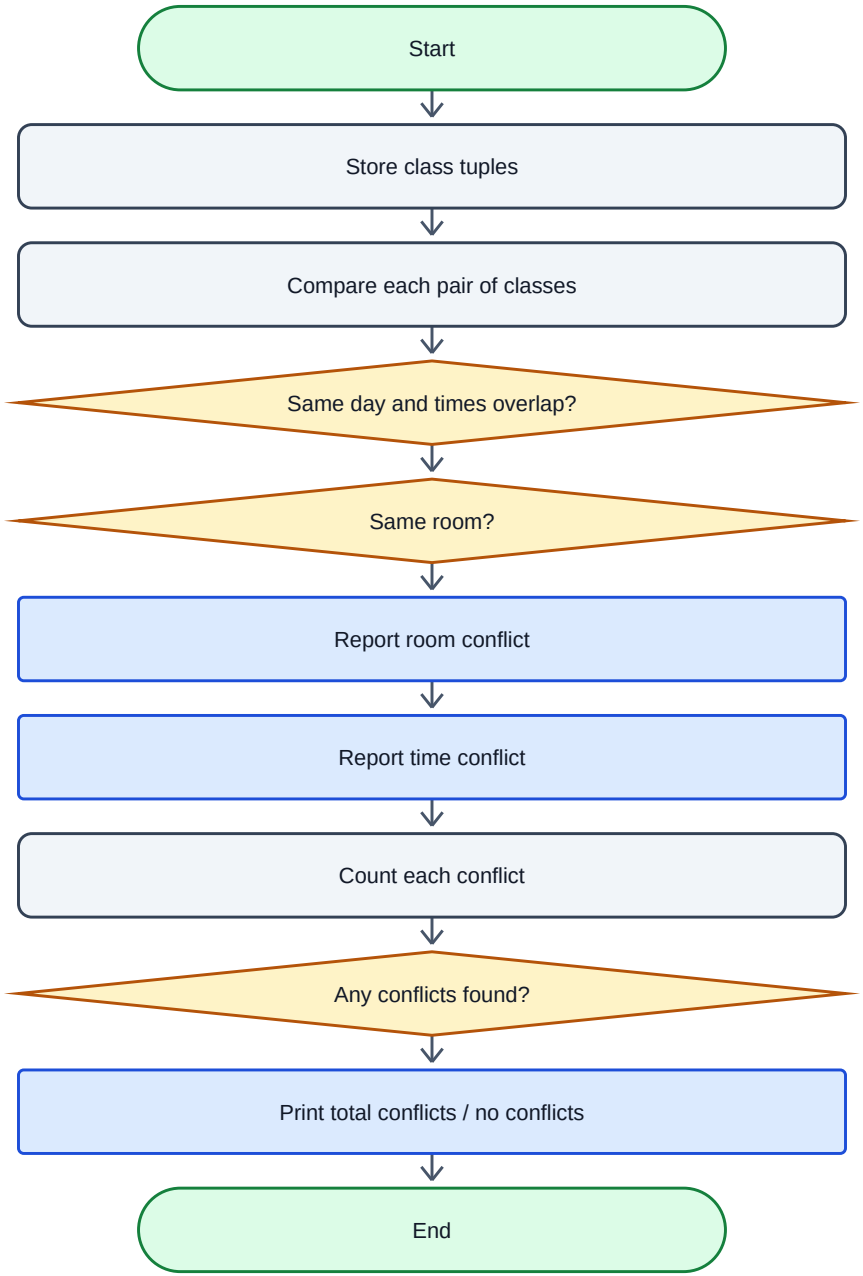
```
TIMETABLE CONFLICT CHECK
Time conflict: Python and Physics overlap on Monday
Total conflicts: 1
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 6

Student Timetable Conflict Detection



Decision branches are represented by the labelled outcomes in the flow.

Student Marks Matrix Analysis

Problem statement

Calculate student totals, averages by subject, and identify the top performer from a nested list.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store marks in a nested list.

2. 2. Calculate each student's total and average.

3. 3. Calculate each subject's average using nested loops.

4. 4. Find the highest total.

5. 5. Display the results.

Python Source Code

```
# Case Study 7: Student Marks Matrix Analysis
# Rows are students; columns are subjects: Python, Maths, Physics.

names = ["Asha", "Balu", "Charan"]
subjects = ["Python", "Maths", "Physics"]
marks = [
    [85, 78, 90],
    [72, 88, 81],
    [91, 93, 89],
]

print("STUDENT TOTALS AND AVERAGES")
totals = []
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)
    print(f"{names[i]}: Total={total}, Average={average:.2f}")

print("\nSUBJECT AVERAGES")
for j in range(len(subjects)):
    total = 0
    for i in range(len(marks)):
        total += marks[i][j]
    print(f"{subjects[j]}: {total / len(marks):.2f}")

top_index = totals.index(max(totals))
print(f"Top student: {names[top_index]} with {totals[top_index]} marks")
```

Sample Output

```
STUDENT TOTALS AND AVERAGES
Asha: Total=253, Average=84.33
Balu: Total=241, Average=80.33
Charan: Total=273, Average=91.00

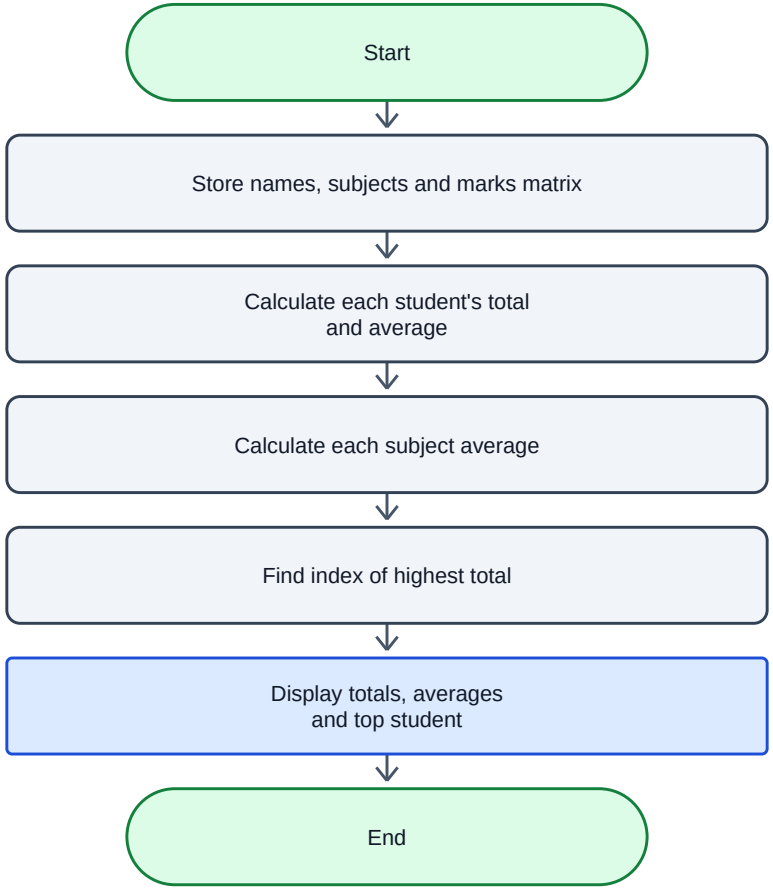
SUBJECT AVERAGES
Python: 82.67
Maths: 86.33
Physics: 86.67
Top student: Charan with 273 marks
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 7

Student Marks Matrix Analysis



Decision branches are represented by the labelled outcomes in the flow.

Sports Tournament Ranking System

Problem statement

Calculate team points and rank teams by points, using goal difference to break ties.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store team results in tuples.

2. 2. Calculate points as wins times three plus draws.

3. 3. Create a ranking list.

4. 4. Sort by points and goal difference in descending order.

5. 5. Print the rankings.

Python Source Code

```
# Case Study 8: Sports Tournament Ranking System
# Points policy: win = 3, draw = 1, loss = 0.
# Results are given as (team, wins, draws, goal_difference).

teams = [
    ("Tigers", 3, 1, 7),
    ("Eagles", 3, 1, 5),
    ("Lions", 2, 2, 4),
    ("Bears", 1, 1, -2),
]

ranked = []
for name, wins, draws, goal_difference in teams:
    points = wins * 3 + draws
    ranked.append((name, points, goal_difference))

ranked.sort(key=lambda team: (team[1], team[2]), reverse=True)

print("TOURNAMENT RANKINGS")
for rank, (name, points, gd) in enumerate(ranked, start=1):
    print(f"{rank}. {name} - Points: {points}, Goal difference: {gd}")
```

Sample Output

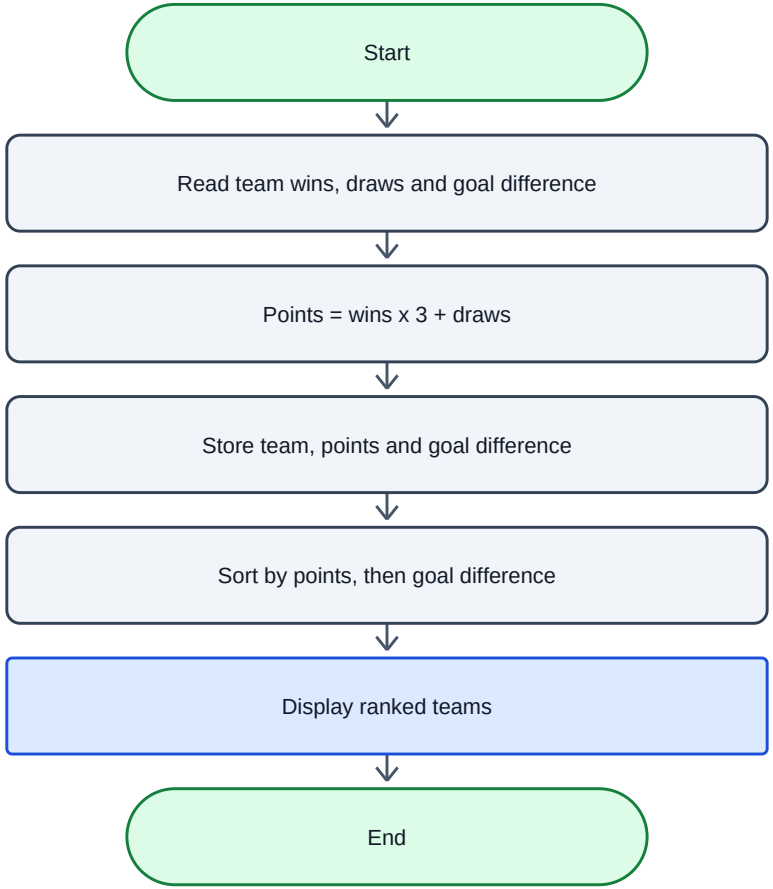
```
TOURNAMENT RANKINGS
1. Tigers - Points: 10, Goal difference: 7
2. Eagles - Points: 10, Goal difference: 5
3. Lions - Points: 8, Goal difference: 4
4. Bears - Points: 4, Goal difference: -2
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 8

Sports Tournament Ranking System



Decision branches are represented by the labelled outcomes in the flow.

Student Academic Risk Classification

Problem statement

Classify students into high, medium, or low academic risk using marks, attendance, and lab scores.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store marks, attendance, and lab score for each student.

2. 2. Calculate the average marks.

3. 3. Check high-risk conditions first, then medium-risk conditions.

4. 4. Classify all remaining students as low risk.

5. 5. Print a reason-relevant summary.

Python Source Code

```
# Case Study 9: Student Academic Risk Classification
# Sample policy: high risk if attendance < 65 or average marks < 40;
# medium risk if attendance < 75 or average marks < 50; otherwise low risk.

students = [
    ("Asha", [65, 70, 72], 90, 85),
    ("Balu", [35, 42, 38], 68, 70),
    ("Charan", [48, 55, 46], 82, 75),
    ("Divya", [60, 62, 58], 60, 65),
]

print("ACADEMIC RISK REPORT")
for name, marks, attendance, lab_score in students:
    average = sum(marks) / len(marks)
    if attendance < 65 or average < 40 or lab_score < 40:
        risk = "HIGH"
    elif attendance < 75 or average < 50 or lab_score < 50:
        risk = "MEDIUM"
    else:
        risk = "LOW"
    print(f"{name}: Average={average:.2f}, Attendance={attendance}%, Lab={lab_score}, Risk={risk}")
```

Sample Output

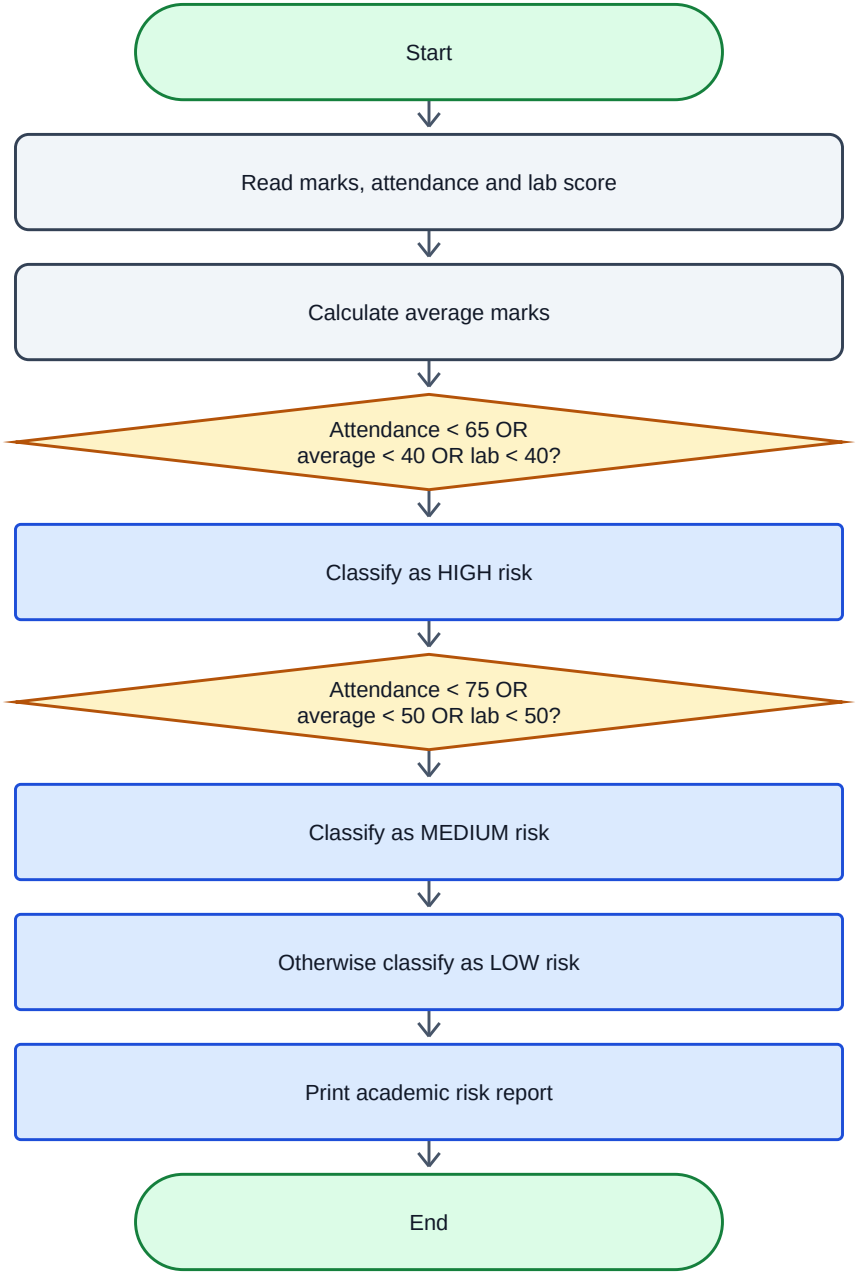
```
ACADEMIC RISK REPORT
Asha: Average=69.00, Attendance=90%, Lab=85, Risk=LOW
Balu: Average=38.33, Attendance=68%, Lab=70, Risk=HIGH
Charan: Average=49.67, Attendance=82%, Lab=75, Risk=MEDIUM
Divya: Average=60.00, Attendance=60%, Lab=65, Risk=HIGH
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 9

Student Academic Risk Classification



Decision branches are represented by the labelled outcomes in the flow.

E-Commerce Order Processing and Reconciliation

Problem statement

Process orders, calculate outstanding quantities, account for returns, and reconcile revenue.

Objectives

Solve the stated problem using Python fundamentals and produce a clear report.

Assumptions

The values and policy rules in main.py are sample data created for demonstration. Replace them if your faculty provides different rules.

Algorithm

1. 1. Store each order as a tuple.

2. 2. Calculate outstanding quantity.

3. 3. Validate dispatched and returned quantities.

4. 4. Calculate net sold quantity and revenue.

5. 5. Print each order and total revenue.

Python Source Code

```
# Case Study 10: Integrated E-Commerce Order Processing and Reconciliation
# Each order: (order_id, product, ordered_qty, dispatched_qty, returned_qty, unit_price)

orders = [
    ("O101", "Headphones", 5, 5, 1, 800),
    ("O102", "Keyboard", 8, 6, 0, 650),
    ("O103", "Mouse", 10, 10, 2, 400),
]

total_revenue = 0
print("ORDER RECONCILIATION")
for order_id, product, ordered, dispatched, returned, price in orders:
    outstanding = ordered - dispatched
    if outstanding < 0:
        print(f"{order_id}: Invalid record - dispatched quantity exceeds ordered quantity")
        continue
    net_sold = dispatched - returned
    if returned > dispatched:
        print(f"{order_id}: Invalid record - returns exceed dispatch")
        continue
    revenue = net_sold * price
    total_revenue += revenue
    print(f"{order_id} {product}: Outstanding={outstanding}, Net sold={net_sold}, Revenue=Rs.{revenue}")

print(f"Total revenue after returns: Rs.{total_revenue}")
```

Sample Output

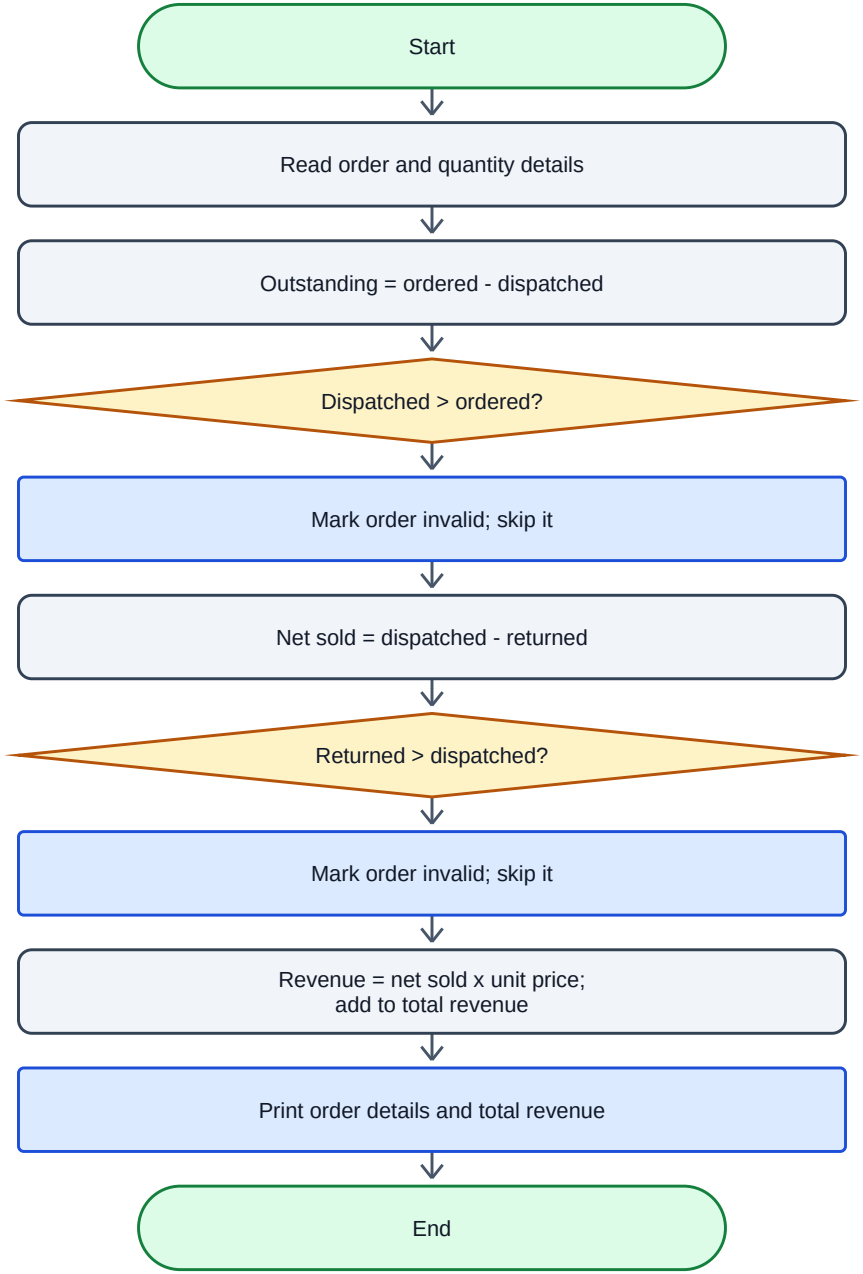
```
ORDER RECONCILIATION
O101 Headphones: Outstanding=0, Net sold=4, Revenue=Rs.3200
O102 Keyboard: Outstanding=2, Net sold=6, Revenue=Rs.3900
O103 Mouse: Outstanding=0, Net sold=8, Revenue=Rs.3200
Total revenue after returns: Rs.10300
```

Conclusion

The program processes the sample data according to the stated rules and displays the result. Run it and capture your own output screenshot for submission.

FLOWCHART — CASE STUDY 10

E-Commerce Order Processing and Reconciliation



Decision branches are represented by the labelled outcomes in the flow.